

상장폐지 대난투 AI 활용 기술 문서

게임 안의 AI · 게임을 만든 AI · AI 산출물을 믿지 않기 위한 장치

0. 요약 — AI가 놓인 세 자리

이 프로젝트에서 AI는 “거들었다”가 아니라 세 자리에 구조적으로 놓여 있다. 각 자리마다 지켜야 할 제약이 다르고, 그 제약이 설계를 결정했다.

자리	무엇을 하는가	그래서 무엇을 지켜야 했나
① 게임 기믹	플레이어가 쓴 한 문장을 해석해 판을 바꾼다 (프롬프트 오브)	심사자가 API 키 없이 실행할 수 있어야 한다 → 해석기를 갈아 끼울 수 있는 계약 으로 설계
② 봇 행동	CPU 셋이 각자 자기 캐릭터의 사용법대로 싸운다	봇이 강해지는 것이 아니라 사용법을 가르쳐야 한다
③ 제작 공정	코드·기획·아트를 생성형 AI로 만들고, 그 공정 자체를 코드로 굳혔다	손으로 반복하는 단계를 남기지 않는다 — 사람이 판단할 것만 사람에게 남긴다

1. 게임 안의 AI ① — 프롬프트 오브

무엇인가

대난투 장르에서 판을 뒤집는 장치는 보통 “스매시볼” 같은 아이템이다. 이 게임은 그 자리에 **프롬프트 오브**를 놓았다. 오브를 깬 사람이 문장을 입력하면, 그 문장이 **아이템 · 맵 · 룰** 중 하나로 해석되어 판이 통째로 바뀐다.

떠다니는 오브를 때려 깬다 → 깬 사람이 문장을 쓴다 → 판이 바뀐다

오브는 체력이 있어 한 대로 깨지지 않는다. 누가 **마지막 타를 넣느냐**가 곧 “다음 규칙을 누가 정하느냐”가 되므로, 자연어 입력이 장식이 아니라 **전투의 목표**가 된다.

핵심 설계 — “문장 → GimmickSpec” 이라는 계약

이 기믹에서 가장 중요한 결정은 **해석기를 인터페이스 뒤로 숨긴 것**이다. 게임의 나머지 전부는 아래 한 줄만 알면 된다.

```
interpretPrompt(raw: string): GimmickSpec
```

`GimmickSpec`은 “무엇이 일어나는가”를 기술한 데이터다 (갈래 · 이름 · 지속시간 · 색 · 아이콘). 이 계약이 있는 한 **해석기가 로컬 규칙이든 LLM 호출이든 게임은 달라지지 않는다**.

왜 지금은 LLM을 부르지 않는가

대회 제출 요건은 “심사자가 별도 유료 라이선스 없이 실행 가능해야 함”이다. 브라우저에서 LLM을 부르려면 API 키가 필요하고, 키를 클라이언트에 넣으면 공개 저장소에 키를 올리는 것과 같다. 그래서 **기본 해석기는 키 없이 도는 로컬 규칙**으로 두고, LLM은 같은 계약을 구현하는 **교체 가능한 대안**으로 남겼다. “AI를 썼다”를 보여주려고 심사자가 못 켜는 게임을 만드는 것은 앞뒤가 바뀐 선택이라고 판단했다.

해석 규칙 — 3단계

`src/config/gimmicks.ts`. 14종의 기믹이 각자 **자기를 부르는 말들**(keywords)을 들고 있고, 입력 문장을 아래 순서로 판정한다.

1. **키워드 점수** — 몇 개가 맞았는지를 먼저 보고, 길이는 동점을 가르는 데만 쓴다. 길이를 먼저 보면 ‘뿌려’ 같은 범용어가 ‘폭탄’ 같은 구체어를 이겨 엉뚱한 기믹이 나간다.
2. **갈래만 지목한 경우** — “아이템 좀 줘”처럼 갈래만 있으면 그 갈래 안에서 **문장 해시**로 고른다.
3. **그래도 못 알아들으면** — 전체에서 문장 해시로 고른다. **아무 일도 일어나지 않는 경우는 없다**.

한국어 자연어 입력을 다루면서 실제로 부딪힌 문제와 대응:

문제	대응
“한 방”과 “한방”이 다른 문자열로 취급된다	띄어쓰기를 뺀 판본도 함께 대조한다
못 알아들으면 아무 일도 안 일어나 “고장난 줄” 안다	미인식 입력도 반드시 하나의 기믹으로 떨어뜨린다
같은 문장에 매번 다른 결과가 나오면 운으로 느껴진다	난수가 아니라 문장 해시 로 고른다 — 같은 문장은 늘 같은 결과
범용어가 구체어를 이겨 의도와 다른 기믹이 나간다	‘뿌려’·‘drop’ 같은 범용 키워드를 카탈로그에서 제거

이 해석기는 브라우저 없이 단독으로 검사한다 — `npm run test:prompt`가 키워드 27건, 미인식 입력 7건, 화면에 노출된 예시 문장 16건, 카탈로그 정합성을 확인한다.

LLM으로 갈아 끼우려면

`interpretPrompt` 한 함수만 교체하면 된다. 게임 쪽 수정은 없다. LLM에게는 **자유 서술이 아니라 카탈로그 안에서 고르게** 해야 한다 — 게임이 실행할 수 없는 결과를 만들어 내면 그 순간 판이

멈추기 때문이다. 실제 전환 시 사용할 지시문 형태는 다음과 같다.

[역할] 너는 대난투 게임의 진행자다.

[입력] 플레이어가 외친 한 문장

[할 일] 아래 카탈로그에서 그 문장에 가장 가까운 기믹 id 를 하나만 고른다.

[카탈로그] item_rain(아이템 폭우) / item_bomb(폭탄 투하) /
map_moon(저중력) / map_lava(용암 지대) / rule_rhythm(리듬 배틀) ... 14종

[규칙]

- 반드시 카탈로그 안의 id 하나만 출력한다. 설명·따옴표·문장을 붙이지 않는다
- 어느 것에도 해당하지 않으면 가장 가까운 것을 고른다. "없음"은 출력하지 않는다
- 문장이 폭력적·부적절해도 게임 기믹으로만 해석한다

게임 쪽은 돌려받은 id를 카탈로그에서 찾고, 못 찾으면 지금의 로컬 규칙으로 되돌아간다. 모델이 죽어도 게임은 멈추지 않아야 한다.

봇도 문장을 외친다

오브는 AI 봇도 깬다. 그때 아무 일도 일어나지 않으면 네 명 중 세 명이 깬 때 이 기능이 아예 보이지 않는다. 그래서 봇이 깨면 **미리 준비된 문장 중 하나를 외치고**, 그 문장이 사람이 입력했을 때와 **완전히 같은 해석기**를 지난다. (“중력을 없애버려”, “리듬에 맞춰 싸우자”, “조작을 뒤집어” 등 10종)

2. 게임 안의 AI ② — 봇에게 캐릭터 사용법을 준다

문제

캐릭터마다 고유 메커니즘을 만들어 놔는데 **봇은 아무도 그것을 쓰지 않았다**. 게이츠 봇은 지분이 1도 안 쌓인 채로 블루스크린을 던지고, 리누스 봇은 방어를 한 번도 하지 않아 ‘포크’를 영원히 못 썼다. 그러면 네 명이 붙어도 사실상 **이름표만 다른 같은 봇 셋**과 싸우는 것이고, 캐릭터를 고를 이유가 절반 사라진다.

대응 — 성격표를 고유 메커니즘에 붙인다

`src/config/aiPersona.ts`. 봇의 성격을 캐릭터 이름이 아니라 **고유 메커니즘**에 붙였다. 이 게임에서 메커니즘이 곧 캐릭터의 정체성이므로 1:1로 대응하고, **새 캐릭터를 넣어도 메커니즘만 정하면 성격이 따라온다**.

상대	봇이 하는 짓
빌 게이츠맨 (지분)	지분을 4개까지 모았다가 블루스크린을 터뜨린다
스티브 잡스맨 (원 모어 씹)	스킬을 맞으면 열린 후속 창에 곧바로 한 번 더 지른다
일론 머스크맨 (부스터)	공중 대시로 붙었다 뺀다. 장외에서도 부스터로 복귀한다
리누스 토발즈맨 (포크)	피하는 대신 막는다 . 막아서 훔친 기술을 되돌려준다
패니 와이즈맨 (풍선)	풍선을 벌고 나면 맞기를 꺼려 도망 다닌다

목표는 “강한 봇”이 아니다. 맞는 쪽에서 “아, 저 캐릭터는 저렇게 쓰는 거구나”를 배우게 하는 것이다. 튜토리얼을 붙이는 대신 **상대의 행동이 곧 설명**이 되게 했다. 그래서 봇의 판단은 LLM이 아니라 결정적인 유한상태기계(FSM)로 만들었다 — 매 판 재현되고, 검사할 수 있고, 네트워크가 없어도 돌아야 하기 때문이다.

성격이 실제로 갈리는지는 `npm run test:ai`가 검사한다 (다섯 갈래가 서로 다른 조건으로 스킬을 쓰는가 · 방어·공중 대시·후속 입력이 갈리는가). 캐릭터는 스무 명이지만 메커니즘은 다섯 갈래이므로, **상대 셋은 서로 다른 갈래에서 뽑는다** — 무작위로만 두면 봇 셋이 같은 방식으로 싸우는 판이 생긴다.

3. 제작 공정의 AI ① — 코드와 설계

Claude Code

게임 코드 전체(약 11,700줄 TypeScript), 시스템 설계, 도구 스크립트, 테스트, 문서를 **Claude Code**와 함께 작성했다. 커밋 이력에 작업 단위가 그대로 남아 있어 (`git log`) 무엇을 어떤 순서로 만들었는지 추적할 수 있다.

작업을 지시할 때 지켰던 원칙 — 이것이 코드 품질을 좌우했다:

- “무엇을”이 아니라 “왜”를 준다. “대시를 추가해줘”가 아니라 “4인 난투에서 서로 밀치기만 하게 된다. 거리 싸움이 생기게 해줘”라고 요구한다. 그래야 맵 확장·카메라 추적까지 함께 나온다.
- 판단 근거를 주석으로 남기게 한다. 이 저장소의 주석은 코드가 하는 일이 아니라 왜 그렇게 정했는지를 적는다. (예: “W는 점프가 아니라 방향키다 — W가 점프를 겹하면 지상 상단기를 낼 방법이 없어진다”)
- 고칠 때마다 근거를 남긴다. 되돌리려는 사람이 같은 실수를 반복하지 않게 하기 위해서다.

4. 제작 공정의 AI ② — 아트 파이프라인

캐릭터·배경·UI 그림은 전부 **Google Gemini** 이미지 생성 (통칭 나노바나나)으로 만들었다. 중요한 것은 “AI로 그림을 뽑았다”가 아니라, 그 공정을 통째로 코드로 굳혔다는 점이다.

<code>npm run prompts</code>	① 프롬프트 46개 생성 (캐릭터 35 + 스테이지·UI 11)
<code>npm run art</code>	② 그 프롬프트로 이미지 생성 → 게임이 읽는 자리에 저장
<code>npm run art:in</code>	②' (무료 경로) 웹에서 받은 그림을 제자리로 정리
<code>npm run sheet:merge</code>	③ 묶음 이미지들을 스프라이트 시트 한 장으로 합침
<code>npm run sheet</code>	④ 시트 전처리 (칸 나누기 · 배경 제거 · 라벨 제거)
<code>npm run art:opt</code>	⑤ 배포 무게로 줄임 (20.6MB → 1.5MB)

① 프롬프트를 코드가 만든다

46장의 프롬프트를 손으로 쓰지 않는다. `tools/gen-prompts.mjs`가 게임 데이터(캐릭터 설정·기술 이름·스테이지 목록)를 읽어 생성한다. 캐릭터를 추가하면 그 캐릭터의 프롬프트가 자동으로 따라 나온다. 기술 이름이 바뀌면 프롬프트의 포즈 설명도 함께 바뀐다 — 데이터와 그림 지시가 어긋날 수 없다.

실제 사용한 프롬프트 (스테이지 배경, 발체)

2D 대난투 게임의 스테이지 배경을 그려줘.

"증권 거래소" - 기본 스테이지 - 전투가 시작되는 곳

[세계관] 주식 시장을 소재로 한 우스꽝스러운 대난투 게임.

배경에는 주가 캔들차트, 전광판, 숫자, 화살표 같은 증권가 요소가 은근히 깔려 있다.

[장면]

한밤중의 거대한 증권 거래소 내부.

벽면을 가득 채운 전광판에 초록·빨강 캔들차트가 흐르고, 숫자가 깜빡인다.

천장은 높고 어둡다. 유리창 너머로 도시의 야경이 흐릿하게 보인다.

[필수 규칙]

- 가로로 매우 긴 이미지. 가로:세로 = 8:3 비율 (예: 1920x720)
- **캐릭터·사람·동물을 그리지 말 것.** 배경만 필요하다
- 글자·로고·워터마크를 넣지 말 것
- 화면 아래 1/3은 캐릭터가 올라설 자리다. 복잡한 무늬 없이 비교적 단순하게
- 가운데는 특히 비워둘 것 - 전투가 벌어지는 자리다
- 전체적으로 **어둡게**. 밝은 캐릭터가 위에 올라가면 대비로 살아난다
- 픽셀아트풍 또는 두꺼운 외곽선의 만화풍. 사진 같은 사실주의 금지

프롬프트에서 배운 것 — “예쁜 그림”을 요구하면 게임에 못 쓰는 그림이 온다. **게임의 제약을 먼저 적어야 한다.** “아래 1/3을 비워라”(캐릭터가 설 자리), “가운데를 비워라”(전투 자리), “어둡게”(밝은 캐릭터와의 대비), “글자를 넣지 마라”(그리면 지워야 한다). 이 네 줄이 쓸 수 있는 그림과 못 쓰는 그림을 갈랐다.

③④ 생성기의 한계를 코드가 흡수한다

42프레임짜리 스프라이트 시트를 한 장에 그리라고 하면 **그림이 무너진다**. 그래서 **6칸씩 7묶음**으로 끊어 뽑고, 받은 이미지들을 코드가 시트 한 장으로 합친다. 전처리 단계에서 다음을 자동으로 흡수한다:

- 투명 배경을 못 만드는 생성기 → 마젠타(#FF00FF)로 받아 게임에서 자동 누끼
- 시키지 않은 칸 구분선·라벨을 그려 넣는 경우 → 검출해 제거
- 묶음마다 다른 크기 → 하나의 규격(124×256)으로 정규화
- 규격(V1 15칸 / V2 30칸 / V3 42칸)을 **프레임 수를 보고 자동 판별**

그 결과 새 시트를 `public/sprites/`에 덮어쓰는 것만으로 캐릭터가 알아 끼워지고, 옛 규격 시트도 그대로 돌아간다 (없는 포즈는 가장 가까운 그림으로 대체).

⑤ 생성형 아트를 웹에 올릴 수 있는 무게로

생성기가 주는 것은 무손실 PNG다. 배경 한 장이 6MB이고 그대로 올리면 심사자의 브라우저에서 로딩만 수십 초가 걸린다 — 그 시점에 “**안 켜지는 게임**”과 같은 말이 된다. `npm run art:opt`가 해상도는 그대로 두고 WebP로만 바꿔 **20.6MB → 1.5MB**, 배포본 전체로는 **32MB → 2.9MB**로

줄인다. 게임은 WebP가 있으면 그쪽을, 없으면 PNG를 읽으므로 “폴더에 넣기만 하면 붙는다”는 작업 흐름은 그대로다.

그림이 없어도 게임은 끝까지 돌아간다

생성형 아트는 한 번에 다 나오지 않는다. 한 장 뽑고 돌려보고, 마음에 안 들면 다시 뽑는다. 그 사이에도 게임이 멈추지 않아야 작업이 이어진다. 그래서 모든 외부 그림은 “있으면 덮어쓰는” 선택 사항이고, 없는 자리는 코드로 그린 화면이 그대로 대신한다. 사운드도 파일 없이 Web Audio API로 실시간 합성한다.

5. AI 산출물을 믿지 않기 위한 장치

AI가 만든 코드와 아트는 **그렇듯하게 틀린다**. 그래서 “돌아가는 것처럼 보이는” 상태를 실제 상태와 구분하는 장치를 함께 만들었다. 이것이 이 프로젝트에서 AI를 쓰는 방식의 절반을 차지한다.

검사	무엇을 잡는가
<code>npm run smoke</code>	타이틀 → 선택 → 전투 → 이동·점프·공격·커맨드 무브·연속기·프롬프트 기믹·스킬·방어·대시 → 결과 경로를 브라우저에서 자동으로 밟고 콘솔 오류를 검사한다 (스크린샷 26장)
<code>npm run test:prompt</code>	자연어 해석기 — 키워드·미인식 입력·화면 노출 예시·카탈로그 정합성
<code>npm run test:ai</code>	봇 성격 다섯 갈래가 실제로 갈리는가
<code>npm run test:sheet</code>	묶음 7장 → 시트 1장 합치기, 규격 자동 판별, 일부 묶음만 뽑은 시트
<code>npm run test:char</code>	로스터 정합성 — 명단 네 곳(파일·타입·배열·그림표)이 일치하는가 · 기술 이름 280개에 중복이 없는가 · 빈 자리가 남았는가
<code>npm run balance</code>	스무 명을 공격·기동·생존·사거리 네 축으로 재고, 2.5σ 를 넘어 혼자 튀는 캐릭터가 있으면 실패시킨다

스무 명을 사람이 다 해 볼 수는 없다

다섯 명일 때는 다 붙여 보고 감으로 맞출 수 있었다. 스무 명이면 맞붙는 조합만 190가지다. 그런데 이 게임의 수치는 전부 데이터이므로 — 기술마다 피해·선딜·판정·후딜이 있으니 초당 피해가 계산되고, 패시브에는 기대 배율이, 스탯에는 무게가 있다 — **플레이 없이도 “누가 혼자 튀는가”는 계산된다**. 이 검사가 실제로 다섯 명을 잡아냈고 (최종량·최고 흡수·최장 사거리가 한 캐릭터에 겹친 경우 등) 수정 후 전체 폭이 3.1σ 안으로 들어왔다. 다만 이 표는 모델이지 승률이 아니다 — 판정 모양·봇 성격·맵은 계산에 없다.

그림으로 단정할 수 없는 것은 상태로 대조한다

스크린샷만으로는 “W+K를 눌렀는데 정말 상승기가 나갔는지”를 단정할 수 없다. 그래서 커맨드 무브는 **실제로 발동한 기술 이름**으로, 프롬프트 기믹은 “**입력한 문장 → 실제로 걸린 룰·중력값**”으로 확인한다. 기대값은 게임 데이터에서 그대로 읽으므로 캐릭터를 추가해도 테스트는 손대지 않는다.

6. 사용한 AI 도구 목록

도구	용도	비고
Claude Code (Anthropic)	게임 코드 · 시스템 설계 · 도구 스크립트 · 테스트 · 문서	프로젝트 전 기간. 커밋 이력에 공동 작성자로 기록 (<code>Co-Authored-By</code>)
Google Gemini 이미지 생성 (나노바나나)	캐릭터 스프라이트 시트 · 스테이지 배경 · UI 그림	모델 <code>gemini-3.1-flash-image</code> (기본값). 웹 UI와 API 두 경로 모두 사용. API 연동 코드: <code>tools/gemini.mjs</code>

게임 실행 중에는 어떤 외부 AI 서비스도 호출하지 않는다. 네트워크가 끊겨도, API 키가 없어도 게임은 완전히 동일하게 동작한다. AI 도구는 제작 시점과 교체 가능한 기믹 해석기 자리에만 놓여 있다.

7. 외부 에셋 · 오픈소스 출처 및 라이선스

오픈소스 라이브러리

이름	라이선스	용도
Phaser 3 (^3.90.0)	MIT	게임 엔진 (런타임 의존성 · 배포본에 포함)
Vite (^7.1.0)	MIT	빌드 · 개발 서버 (개발 전용)
TypeScript (^5.9.2)	Apache-2.0	언어 · 타입 검사 (개발 전용)
Playwright (^1.62.1)	Apache-2.0	스모크 테스트 · 이미지 최적화 · PDF 생성 (개발 전용)
pngjs (^7.0.0)	MIT	스프라이트 시트 전처리 (개발 전용)

런타임에 실제로 배포되는 서드파티 코드는 **Phaser 3 (MIT)** 하나뿐이다. 이 프로젝트 자체의 라이선스는 MIT다.

그래픽

대상	출처
캐릭터 스프라이트 5종	Google Gemini 이미지 생성으로 이 프로젝트를 위해 새로 제작. 기존 저작물 파일을 가져다 쓰지 않았다. 프롬프트 원문은 <code>art-source/prompts/</code> 에 전부 포함
스테이지 배경 3종	동일 (증권 거래소 · 정전 · 달 표면). 원본 PNG와 프롬프트를 저장소에 보관
이펙트 · 아이템 아이콘 · UI · 파티클	외부 파일 없음 — 전부 코드로 그린다 (Phaser 도형 / 이모지)
폰트	시스템 폰트만 사용 (Malgun Gothic · Apple SD Gothic Neo · Noto Sans KR). 폰트 파일을 배포본에 포함하지 않는다

사운드

음원 파일이 하나도 없다. 타격음·점프음·BGM까지 전부 Web Audio API로 실시간 합성한다 (`src/systems/SoundSystem.ts`). 따라서 사운드 관련 외부 저작물 이용이 없다.

캐릭터 소재에 관하여

등장 캐릭터는 IT 업계 공인(公人)과 대중문화 캐릭터를 소재로 한 패러디 창작 디자인이며, 이름 또한 원본과 구별되도록 변형했다(빌 게이츠맨 · 스티브 잡스맨 · 일론 머스크맨 · 리누스 토발즈맨 · 패니 와이즈맨). 모든 이미지는 기존 저작물을 복제·가공한 것이 아니라 생성형 AI로 새로 제작했다.